Aula 009 Vinculação estática e dinâmica

Enunciado: Para cada trecho de código: (a) indique a saída; (b) informe se a decisão ocorre na compilação ou na execução.

Exercício 1 Java

```
class Animal {
    String som() { return "..."; }
}
class Cachorro extends Animal {
    String som() { return "au"; }
}
Animal x = new Cachorro();
System.out.println(x.som());
```

Saída	Momento da decisão
au	Em execução (vinculação dinâmica).

Explicação: A variável **x** foi declarada como **Animal**, mas referencia um objeto **Cachorro**. Como **som()** é um método de instância sobrescrito, a JVM seleciona em tempo de execução a implementação do objeto real: **Cachorro.som()**.

Exercício 2 C++

```
struct A {
    void f() { cout << "A"; }
};
struct B : A {
    void f() { cout << "B"; }
};
B b;
A *p = &b;
p->f();
```

Saída	Momento da decisão
A	Na compilação (vinculação estática).

Explicação: O ponteiro **p** tem tipo declarado **A***. Como **A::f()** não foi declarado com **virtual**, a chamada é vinculada a **A::f()** pelo tipo do ponteiro, mesmo que ele aponte para um objeto **B**.

Exercícios 3 e 4

Enunciado: Para cada trecho de código: (a) indique a saída; (b) informe se a decisão ocorre na compilação ou na execução.

Exercício 3 Java

```
class A {
    String nome = "A";
    String getNome() { return nome; }
}
class B extends A {
    String nome = "B";
    String getNome() { return nome; }
}
A x = new B();
System.out.println(x.nome + " " + x.getNome());
```

Saída	Momento da decisão
`A B`	Mista: campo na compilação; método em execução.

Explicação: Campos não são polimórficos em Java. Assim, **x.nome** é resolvido pelo tipo declarado de **x** (**A**) e produz **A**. Já **x.getNome()** sofre despacho dinâmico: a implementação de **B** é chamada e retorna o campo **nome** de **B**, produzindo **B**.

Exercício 4 Python

```
class Contador:
    total = 0

    def __init__(self):
        Contador.total += 1
        self.id = Contador.total

a = Contador()
b = Contador()
print(a.id, b.id, a.total)
```

Saída	Momento da decisão
`1 2 2`	Em execução (resolução dinâmica de atributos).

Explicação: **total** é um atributo de classe, compartilhado por todas as instâncias. Ao criar **a** e **b**, ele passa de 0 para 1 e depois para 2; por isso os IDs são 1 e 2. Como **a** não possui um atributo de instância chamado **total**, a busca continua na classe e encontra o valor 2.

Exercícios 5 e 6

Enunciado: Para cada trecho de código: (a) indique a saída; (b) informe se a decisão ocorre na compilação ou na execução.

Exercício 5 Java

```
class A {
    static String quem() { return "A"; }
}
class B extends A {
    static String quem() { return "B"; }
}
A x = new B();
System.out.println(x.quem());
```

Saída	Momento da decisão
`A`	Na compilação (método estático).

Explicação: Métodos **static** não são sobrescritos; eles podem apenas ser ocultados. A chamada é escolhida pelo tipo declarado da referência, que é **A**. Logo, o resultado é **A**. Para tornar a intenção explícita, o recomendado é chamar diretamente **A.quem()**.

Exercício 6 Go

```
type Animal struct{}

func (Animal) Som() string { return "..." }
func (a Animal) Falar() string {
    return "faz " + a.Som()
}

type Cao struct{ Animal }
func (Cao) Som() string { return "au" }

fmt.Println(Cao{}.Falar(), Cao{}.Som())
```

Saída	Momento da decisão
`faz ... au`	Na compilação (sem despacho por interface).

Explicação: Embedding não é herança. **Cao{}.Falar()** usa o método promovido de **Animal**; dentro dele, o receptor **a** tem tipo **Animal**, então **a.Som()** chama **Animal.Som()**. A segunda expressão, **Cao{}.Som()**, chama diretamente o método definido em **Cao**. Para despacho dinâmico em Go, seria necessário trabalhar por meio de uma interface.